

gameboy3

Un atelier maison pour écrire, dessiner et faire tourner de vraies cartouches Game Boy — langage, compilateur, émulateur et éditeur, sans dépendance externe.

79

leçons vérifiées
automatiquement

10

niveaux, du premier
programme au jeu
complet

30

exemples fournis, tous
compilés

~8 900

lignes pour le compilateur
et l'émulateur

Ce que c'est

`gameboy3` n'assemble pas des briques existantes : c'est un langage (un C++ volontairement restreint), un compilateur qui l'émet directement en instructions du processeur SM83, un émulateur Game Boy / Game Boy Color écrit à la main, et un éditeur graphique — le tout dans une seule page web, sans RGBDS, sans GBDK, sans émulateur venu d'un paquet npm.

Une cartouche compilée ici démarre sur une vraie console : la sortie n'est pas une approximation pédagogique, c'est un fichier `.gb` ou `.gbc` de 32 Ko, structurellement identique à ceux du commerce.

Ce qui le rend rare

Un langage qui refuse, et qui dit pourquoi

Le compilateur ne traduit jamais en douce ce qu'il ne peut pas garantir : un `float`, une classe, un mauvais nombre d'index sur une grille sont refusés avec le numéro de ligne exact et la correction à faire. La leçon 78 en fait un sujet à part entière — un compilateur qui aide n'est pas seulement celui qui compile, c'est celui qui dit non correctement.

La couleur de la Game Boy Color, sans écrire une ligne

Huit palettes de quatre couleurs, onze variétés et huit thèmes toutes prêtes : on peint une tuile, un personnage ou une carte, et l'atelier écrit lui-même les `couleurFond` et les `teindre` — dans les limites exactes de la console (quatre couleurs par carré de 8×8). Changer une couleur ne touche que le dessin ouvert : une palette partagée est copiée, jamais changée en cachette pour tout l'écran.

Une musique qui ne ralentit jamais

Le séquenceur (`Air`, `jouer()`) avance dans l'interruption VBlank, pas dans la boucle du jeu : un jeu qui calcule beaucoup ne fait jamais traîner sa propre musique. Un atelier à la souris l'écrit, avec écoute immédiate sans recompiler.

De la rétro-ingénierie, honnêtement présentée

Le **MODE MACHINE** désassemble n'importe quelle cartouche compilée ici et retrouve les noms de variables et de fonctions — parce que le compilateur vient de les y ranger. Une cartouche venue d'ailleurs ne montre que des nombres, et l'outil le dit franchement plutôt que de prétendre deviner.

Un tutoriel qui ne peut pas mentir

Rien n'est rédigé à côté du code : chaque leçon est compilée pour de vrai, jouée dans l'émulateur du projet, et les contrôles qu'elle affiche sont ceux que `verifier-tuto.mjs` exige — mesurés, pas racontés. Le PDF (`tutoriel.pdf`, 79 pages) et les 79 livrets détaillés sont fabriqués depuis les mêmes leçons, jamais recopiés à la main.

En pratique : modifier une leçon et oublier de relancer la vérification est *détecté*, pas seulement possible — `verifier-tutoriels.mjs` régénère la documentation en mémoire et refuse si elle diffère de ce qui est commité.

Comparé à l'ambition de départ (« éгалer PICO-8 »)

PILIER	PICO-8	GAMEBOY3
Mode code	Lua, interprété	C++ restreint, compilé en instructions réelles
Sprites	éditeur intégré	éditeur à la souris, ou en texte
Musique / son	séquenceur + effets	séquenceur indépendant du jeu, + bruitages
Cartouche	format « fantaisie » (.p8)	.gb / .gbc réels , jouables sur console d'origine
Palette / couleurs	un éditeur de palette	un onglet dédié (« 🎨 Les couleurs »)
Bibliothèque de dessins de départ	non (le partage de cartouches, hors éditeur)	oui — un onglet « 🎭 Les modèles », personnages et tuiles tout faits à reprendre
Effets sonores courts, séparés de la musique	oui (<code>sfx()</code> , éditeur dédié)	possible en code (<code>note()</code> / <code>bruit()</code>), pas d'éditeur visuel dédié
Export en HTML autonome	oui	oui — une page qui embarque l'émulateur et la cartouche, jouable en double-cliquant, sans atelier ni serveur ; vérifié automatiquement (<code>verifier-export.mjs</code>)
Carte pilotée par indicateurs de sprite	oui (<code>fget</code> / <code>fset</code>)	pas encore — identifié dans <code>PARCOURS.md</code>

Vérifié en pilotant l'atelier réel (et non relu dans le seul code) : 4 modes (Création, Code, Leçons, Machine), 4 onglets dans Création (tuiles, airs, couleurs, modèles), 28 exemples dans le sélecteur, 53 réglages en 7 groupes.

Coût de reconstruction : une hypothèse, pas la réalité de ce projet

Ce projet-ci a été construit en moins d'un mois, à deux, avec une IA. Le calcul ci-dessous répond à une question différente et purement hypothétique : **combien de temps faudrait-il à UNE SEULE personne, seule, sans aide, tapant et déboguant chaque ligne à la main**, pour produire le même volume ? Les deux chiffres ne se comparent pas — le second sert seulement à donner une idée de l'ampleur du volume produit, pas du temps réellement passé.

Aucune heure n'a été pointée : ce n'est **pas une facture**. La méthode : le volume réel de chaque partie du dépôt, divisé par une vitesse de production prudente *pour un humain seul et non assisté* selon sa nature (le code d'un compilateur se teste et se corrige bien plus lentement qu'une leçon qui suit un modèle déjà établi).

`TUTORIELS.md` n'est pas compté : c'est un fichier engendré depuis `tuto/lecons.js`, déjà comptabilisé — le compter aussi aurait été un double emploi.

NATURE DU TRAVAIL	VOLUME RÉEL	VITESSE RETENUE	JOURS ESTIMÉS
Compilateur + émulateur (le cœur technique)	8 888 lignes	15 à 25 lignes/jour	355 à 593
Interface, éditeurs, outils PDF	10 100 lignes	30 à 45 lignes/jour	224 à 337
Leçons, tests automatisés, exemples, documentation écrite	20 971 lignes	70 à 140 lignes/jour	149 à 300
Total	39 959 lignes	—	728 à 1 230 jours

Taux journalier à appliquer : € / jour

Hypothèse solo, non assisté : entre 291 200 € et 492 000 € (728 à 1 230 jours × 400 €/j), soit l'équivalent d'environ **3,3 à 5,6 années** de travail à temps plein d'une seule personne (220 jours ouvrés/an) — **contre moins d'un mois, dans les faits, à deux avec une IA**.

Réserves à garder en tête : le nombre de lignes est un indicateur grossier de l'effort réel — il ne dit rien du temps passé à concevoir le langage, à choisir la pédagogie du tutoriel, ou à corriger un bogue invisible pendant des heures. Les vitesses retenues sont des repères de l'industrie du logiciel, pas une mesure prise sur ce projet précis. Et si tout ou partie de ce code a été écrit avec l'aide d'un outil d'IA plutôt que tapé à la main, le temps humain réellement passé peut être largement inférieur à ce que ce calcul suppose.

Où ça peut aller ensuite

`PARCOURS.md` détaille, genre par genre, ce qui est déjà utilisable et ce qui manquerait pour un jeu complet :

- **Déjà praticables de bout en bout** — puzzle (Tetris) et plateforme (Mario), chacun avec son chemin de douze leçons.
- **Presque prêts** — plateforme-tir façon Mega Man : ses briques manquantes (tir du joueur, jauge de vie, invincibilité temporaire) tournent désormais ensemble dans un exemple complet et jouable (`projets/mon_mario`) ; ce qu'il resterait pour en faire un chemin de leçons comme Tetris et Mario — plusieurs types d'ennemis, des salles fixes plutôt qu'un défilement continu.
- **Jamais commencés** — RPG, shoot 'em up, aventure vue du dessus, course, sport, Sokoban, jeu de rythme : chacun a déjà une partie de ses briques (le déplacement, les collisions, le hasard, le son), mais aucun n'a

encore ce qui le rend vraiment jouable.

Document généré depuis l'état réel du dépôt à la date de rédaction. Les chiffres techniques (leçons, contrôles, exemples) sont vérifiés par `npm run verifier` ; le coût de reconstruction, lui, est une **estimation** expliquée ci-dessus, pas une mesure. Détail des sources : `LISEZMOI.md`, `TUTORIELS.md`, `CHANGEMENTS.md`, `PARCOURS.md`.